App Dev 2 Project Report

Student details

Name: Rohan Uday Puntambekar

Roll Number: 21f3002298

Email: 21f3002298@ds.study.iitm.ac.in

I am a BS Degree student at IIT Madras with an interest in artificial intelligence and machine learning research. Apart from pursuing this diploma, I am currently a Master's student in Artificial Intelligence at Northeastern University, Portland, Maine, in the United States. This project was completed as part of the Modern Application Development II course in the January 2026 term.

Project details

Project Title: Placement Portal Application V2

Problem Statement: Institutes need an efficient system to manage campus recruitment activities. The objective of this project is to build a modern web application using a decoupled architecture with Flask REST APIs on the backend and Vue.js on the frontend, along with Redis caching and Celery-based background jobs.

Approach: I started by designing the database schema, which consists of five tables: User, Company, Student, PlacementDrive, and Application. Since the domain logic is the same as the App Dev I project, I focused on restructuring the architecture. The backend was built as a pure REST API layer using Flask, with JWT-based authentication replacing Flask-Login sessions. Each role (admin, company, student) has its own set of API endpoints organized in separate Blueprint files.

The frontend is a Vue.js 3 single-page application loaded via CDN and served through Flask. Vue Router handles client-side navigation, and Axios makes asynchronous HTTP calls to the backend APIs. The JWT token is stored in localStorage and attached to every request via the Authorization header.

After the core features were functional, I integrated Celery with Redis as the message broker for three background jobs: a scheduled daily reminder for upcoming application deadlines, a scheduled monthly placement report generator, and a user-triggered asynchronous CSV export of application history. Redis was also used for API response caching on frequently accessed endpoints, such as the admin dashboard and drive listings, using Flask-Caching.

AI/LLM declaration

I used Claude (Anthropic) to assist in developing this project. The AI was used to debug and refine the Flask REST API structure, and help me properly configure the Celery worker with Redis. I also referred to AI suggestions for structuring the Vue Router and Axios integration patterns. All business logic decisions, runtime testing, end-to-end workflow verification, integration decisions, and bug fixes during testing were completed by me. The estimated extent of AI/LLM usage is around 15-20%.

Technologies and frameworks used

Flask serves as the backend API framework. It handles routing and request processing, returning JSON responses instead of rendered HTML. Routes are organized using Flask Blueprints, with separate modules for auth, admin, company, student, and background job endpoints.

SQLAlchemy (Flask-SQLAlchemy) is the ORM used for database modeling. The five tables and their relationships are defined as Python classes, and all queries are performed through SQLAlchemy's query interface.

PyJWT handles token-based authentication. On login, the server generates a JWT containing the user ID, role, and expiration timestamp. A custom *token_required* decorator extracts and validates this token from the Authorization header on every protected request.

Vue.js 3 (CDN) powers the reactive frontend. All UI components are defined within a single *index.html* file using Vue's options API. Each page (login, dashboards, drive management, etc.) is a separate Vue component.

Vue Router 4 (CDN) provides client-side routing with hash-based navigation. This allows the entire SPA to be served from a single HTML file without server-side routing configuration.

Axios (CDN) is the HTTP client used for all API communication between the Vue frontend and Flask backend. The JWT token is attached to each request header.

Bootstrap 5 (CDN) provides styling for all UI elements, including tables, cards, forms, alerts, and the navigation bar.

SQLite is the database, created programmatically through SQLAlchemy's *db.create_all()*.

Redis (Memurai on Windows) serves two purposes: as the message broker and result backend for Celery tasks, and as the caching layer for Flask-Caching.

Celery is the distributed task queue used for running background jobs. Three tasks are implemented: a daily reminder job, a monthly report generator, and an on-demand CSV export.

Flask-Caching provides Redis-backed response caching. The admin dashboard endpoint is cached with a 60-second timeout to reduce repeated database queries.

Database schema and ER diagram

The database consists of five tables. The schema is shared with the App Dev I project since the domain model is identical.

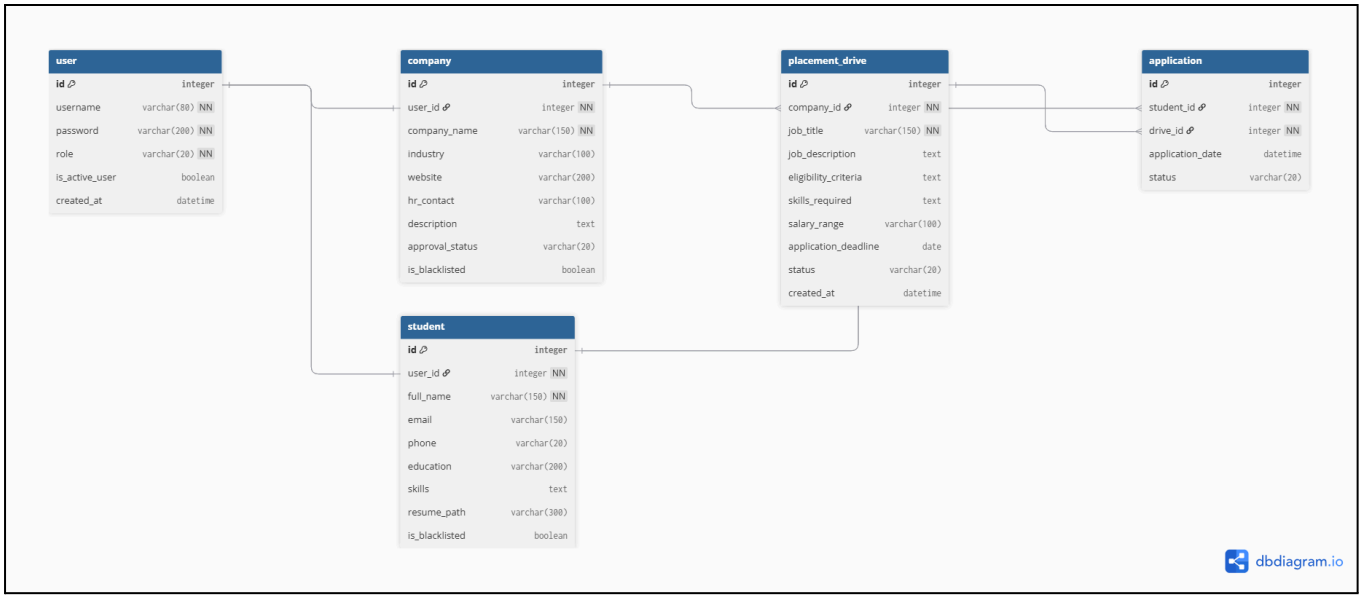
User stores login credentials and a role field (admin, company, or student). It has one-to-one relationships with both the Company and the Student tables.

Company holds the company profile data and links to User via a foreign key. The *approval_status* field controls whether the company can log in and create drives. A company can create many PlacementDrives.

Student holds student profile data. A student can submit many Applications, but not for the same PlacementDrive.

PlacementDrive represents a job posting by a company. It requires admin approval before it becomes visible to students. Each drive can receive many Applications.

Application connects a student to a drive with a status field tracking the progression from applied through shortlisted to selected or rejected. A unique constraint on (student_id, drive_id) prevents duplicate applications at the database level.



API resource endpoints

Authentication: POST /api/login returns a JWT token. POST /api/register creates a new company or student account.

Admin endpoints (require admin JWT): GET /api/admin/dashboard returns summary statistics (cached for 60 seconds). GET /api/admin/companies and GET /api/admin/students support search via query parameter. PUT endpoints at /api/admin/company/{id}/approve, /reject, and /blacklist manage company status. PUT endpoints at /api/admin/student/{id}/blacklist toggle student access. GET /api/admin/drives lists all drives with PUT endpoints for approve and reject. GET /api/admin/applications lists all applications. POST /api/admin/trigger-report and /trigger-reminders dispatch Celery background jobs.

Company endpoints (require company JWT): GET /api/company/dashboard returns company details and drive list. POST /api/company/drives creates a new placement drive. PUT endpoints at /api/company/drive/{id}/close and /reopen manage drive status. GET /api/company/drive/{id}/applications lists applicants for a drive. PUT /api/company/application/{id}/update changes the application status.

Student endpoints (require student JWT): GET /api/student/dashboard returns stats and status change notifications. GET /api/student/drives supports search and indicates which drives the student has already applied to. POST /api/student/apply/{id} submits an application. GET /api/student/applications lists the student's application history. GET and PUT /api/student/profile handle profile retrieval and update. POST /api/student/export-csv triggers an asynchronous CSV export job. GET /api/student/{id}/view allows admin and company roles to view a student profile.

Job status: GET /api/jobs/task-status/{task_id} polls the status of an asynchronous Celery task.

Architecture and features

Project structure: The backend follows a modular Blueprint pattern. *backend/app.py* is the application factory. *backend/models.py* defines the database schema. *backend/extensions.py* initializes SQLAlchemy and Flask-Caching. Route handlers are split across *backend/routes/auth.py*, *admin.py, company.py, student.py, and jobs.py. backend/celery_worker.py* defines the Celery app instance, beat schedule, and all three background tasks. The frontend is a single *frontend/index.html* containing all Vue components.

Core features implemented:

JWT authentication with role-based access control: The *token_required* decorator validates the JWT on every protected endpoint. A *role_required* decorator restricts access to specific roles. Companies cannot log in until approved by the admin. Blacklisted users are blocked at the login endpoint.

The admin dashboard displays aggregate statistics with Redis caching. Admin can approve, reject, and blacklist both companies and students. Drive approval and rejection are handled through dedicated endpoints. Search is supported for both companies and students.

Companies can create placement drives (only after admin approval), manage drive status, view applicants with their profiles and skills, and update application status through a dropdown interface.

Students can browse and search approved drives, apply with duplicate prevention, track application status, view dashboard notifications for status changes, edit their profile, and trigger an asynchronous CSV export of their application history.

Three Celery background jobs are implemented. The daily reminder job queries for deadlines within 3 days and logs reminders for all active students. The monthly report job generates an HTML report with placement statistics and saves it to the static/reports directory. The CSV export job is triggered by the student from the dashboard, generates a CSV file with application history, and the frontend polls the task status endpoint until completion.

Redis caching is applied to the admin dashboard endpoint with a 60-second expiry to reduce database load on repeated requests.

Video presentation

Drive Link: https://drive.google.com/file/d/1fSvM3seMMoCC4AvX0qqkyk6fbjx7WNIT/view?usp=sharing